

CS641 / CS708
Introduction to Computer Vision
Lecture 01: Introduction

Dr. Abdul Basit — Assistant Professor, CSIT, University of Balochistan
May 15, 2024

Topics Covered

- 1. What is Computer Vision?**
- 2. Applications of Computer Vision**
 - 2.1 Professional / Industrial Applications
 - 2.2 Broader Consumer-Level Applications
- 3. Course Information**
 - 3.1 Software Used
 - 3.2 Marks Distribution
 - 3.3 Recommended Readings
 - 3.4 Semester Focus

1. What is Computer Vision?

Computer Vision is the field of artificial intelligence that teaches computers how to see, interpret, and understand images and videos — just like the human eye and brain do. In other words, we want to give a machine the ability to extract meaningful information from visual data (photographs, videos, 3D scans, etc.).

□ Simple Analogy

Think of a camera as a pair of mechanical eyes. On its own, the camera just collects light and stores it as numbers (pixels). Computer Vision is the brain behind those eyes — algorithms that look at those numbers and decide: 'This is a cat', 'That car is 10 metres away', or 'This X-ray shows a fracture'.

1.1 The Vision System Diagram

A vision system takes images as input and uses three foundational disciplines to produce useful information:

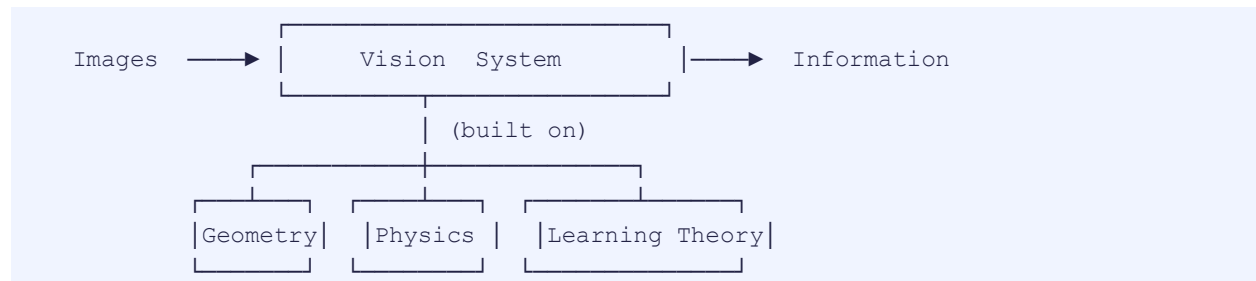


Figure 1 — The three pillars of a Vision System

The three pillars explained:

- Geometry — How do 3D objects in the real world map onto a 2D image? (camera models, perspective projection, stereo vision)
- Physics — How does light interact with surfaces and lenses? (reflectance, illumination, colour)
- Learning Theory — How do we train algorithms to recognise patterns? (machine learning, deep neural networks)

1.2 The Output is Application-Specific

There is no single 'answer' a vision system always produces. What information you extract depends entirely on what you need for your task:

Output Type	Real-World Example
3D Models	Reconstruct a building from multiple photos

Output Type	Real-World Example
Object Poses	Detect the orientation of a robotic arm
Object Categories	Is this photo a cat or a dog?
Camera Poses	Figure out where a camera was when a photo was taken

1.3 Parts of a Vision System

A complete computer vision system has three components:

- Image Acquisition Hardware — the physical device that captures the image:
 - Analog camera + digital frame grabber (older systems)
 - Digital camera with fast serial interface (USB, FireWire, GigE) — modern standard
- Image Processing Support Software — libraries and frameworks (e.g. OpenCV, PIL/Pillow, scikit-image)
- Computer Vision Algorithms — your own code that analyses, detects, classifies, or reconstructs from the image

□ Key Point

The vision system is not just the camera — it is the full pipeline: hardware to capture + software to process + algorithms to understand.

2. Applications of Computer Vision

Computer vision is one of the most widely-applied areas in all of computing. Below are the major application categories, with explanations of how they work.

2.1 Professional and Industrial Applications

Application	How Computer Vision is Used
Optical Character Recognition (OCR)	Converts images of printed or handwritten text into machine-readable text. Used in document scanning, passport reading, and number-plate recognition.
Industrial Inspection & Control	Cameras on assembly lines inspect products for defects (cracks, misalignment, wrong colour) faster and more accurately than human workers.
Mobile Robot Navigation	Robots use stereo cameras or depth sensors to map their environment, detect obstacles, and plan paths. Think: autonomous warehouse robots.
Military Intelligence	Aerial and satellite images are analysed to detect vehicles, troops, infrastructure changes, or unusual activity.
Security / Surveillance	Camera feeds are monitored automatically — detecting intruders, tracking movement, or flagging abandoned bags in airports.
Fingerprint / Biometrics	Fingerprint, iris, and face recognition use vision algorithms to verify identity. Used in smartphones, border control, banking.
Digital Library Image Retrieval	Search engines for images — find all photos that contain a 'red car' or a 'smiling person'.
Medical Image Analysis	Detect tumours in MRI scans, count blood cells in microscopy images, measure bone density in X-rays — often better than human radiologists.
3D Model Capture	Take many photos of an object or scene and reconstruct a 3D model for animation, VR, digital heritage, or engineering.

2.2 Broader Consumer-Level Applications

These are computer vision features that you may already use every day, built into cameras, phones, and apps:

Image Stitching (Panorama)

Multiple overlapping photographs are automatically aligned and blended together to form a single wide-angle (panoramic) image.

[Photo 1] [Photo 2] [Photo 3]



Figure 2 — Image Stitching combines overlapping photos

Exposure Bracketing (HDR Photography)

Several photos of the same scene are taken at different exposure levels (dark, normal, bright). Computer vision algorithms merge them to produce one image that captures detail in both bright and dark regions — just like the human eye sees the world.

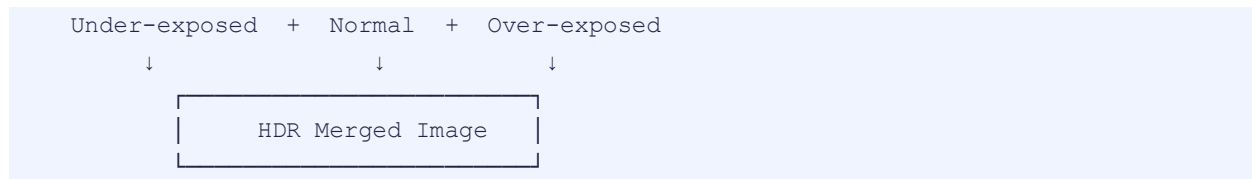


Figure 3 — Exposure Bracketing / HDR Merging

Face Detection

Cameras automatically locate all human faces in an image, even in a cluttered background. This is used for auto-focus, privacy blurring, and access control.

Morphing (Image Blending)

Two photographs are blended smoothly — the computer figures out corresponding points in both images (e.g., eye to eye, mouth to mouth) and creates a gradual transition between them.

[Face A] → 50% blend → [Face B]
(e.g., human face blending into a cheetah face)

Figure 4 — Morphing between two photos

3D Modeling from Photographs

A collection of photos of a building or object, taken from different angles, is turned into a full 3D model. The process involves: (1) finding matching features between photos, (2) computing camera positions, (3) reconstructing 3D point clouds, (4) generating textured meshes.

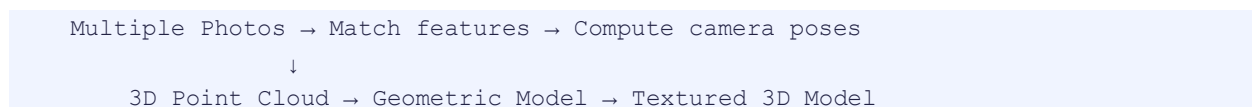


Figure 5 — 3D Model capture pipeline from photographs

Photo-Based Walkthroughs

Using many photographs of a place (e.g. from Google Street View), a virtual walkthrough experience is generated. The viewer can navigate through the scene as if physically present.

Structure from Motion (SfM)

A powerful algorithm that reconstructs a 3D point cloud of a large complex scene from hundreds of partially overlapping photographs. The algorithm simultaneously estimates where each photo was taken (camera pose) and what 3D structure is visible (scene geometry).

Stereo Matching

Uses two cameras (like human eyes) to compute depth — how far away each point in the scene is. By comparing the small differences (disparities) between the left and right camera images, a detailed 3D depth map is produced. Used in self-driving cars and surgical robots.

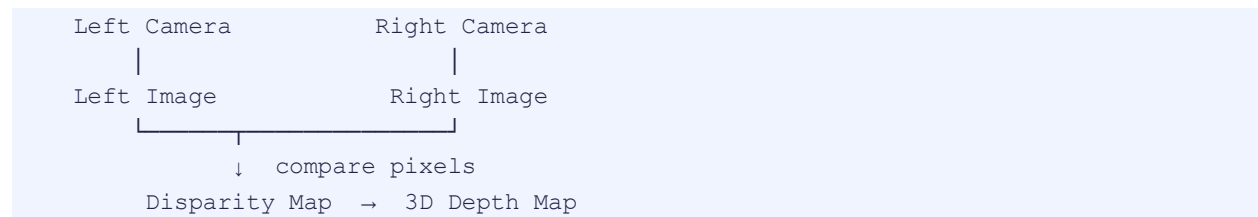


Figure 6 — Stereo Matching uses two cameras to estimate depth

Person Tracking

A moving person (or vehicle, or animal) is followed across video frames. The algorithm detects the person in each frame and links detections across time, even when the person is partially hidden.

3. Course Information

3.1 Software Used in This Course

All practical work uses Python with the OpenCV library:

- Python 3.x — The programming language. Free, widely used in AI/ML/CV.
- OpenCV (Open Source Computer Vision Library) — The industry-standard library for image processing and computer vision. Provides functions for reading images, filtering, edge detection, feature matching, and much more.

□ Installation

Install both with a single command: `pip install opencv-python` For additional scientific computing tools also used in the course: `pip install numpy matplotlib scipy`

3.2 Marks Distribution

Assessment Component	Weight (%)
Mid-Term Examination	30%
Terminal (Final) Examination	40%
Assignments and Quizzes	15%
Research Report	15%

3.3 Recommended Readings

- Computer Vision: Algorithms and Applications — Richard Szeliski (Primary textbook; freely available online at szeliski.org/Book)
- Introduction to Computer Vision — CS143 (Stanford/Harvard course slides and notes)
- Computer Vision lectures — Mubarak Shah (University of Central Florida; available on YouTube)

3.4 Semester Focus

This semester concentrates on three main algorithmic areas:

- 3D Reconstruction — how to recover 3D geometry from 2D images
- Learning — using machine learning to recognise and classify visual content
- Sequential State Estimation / Tracking — following objects across video frames

To achieve this, we begin from fundamentals:

- Understanding how cameras work (projective geometry, camera models)

- Understanding image formation (how 3D scenes become 2D pixels)
- Then using this knowledge to invert the process and reconstruct 3D scenes from images

□ **Big Picture**

The entire course builds toward one goal: giving computers the ability to understand the 3D world from 2D photographs — the same challenge the human visual system solves effortlessly every second of every day.

CS641

Introduction to Computer Vision

Lecture 02: Working with Images

Dr. Abdul Basit — Assistant Professor, CSIT, University of Balochistan
May 15, 2024

Topics Covered

1. Image Basics

- 1.1 Image Types Overview
- 1.2 Binary Images
- 1.3 Grayscale Images
- 1.4 Color Images
- 1.5 Image Histograms

2. Image Noise

- 2.1 What is Noise?
- 2.2 Mathematical Model
- 2.3 Gaussian Noise

3. OpenCV Functions

- 3.1 Sample Program: Read and Display
- 3.2 Convert to Grayscale
- 3.3 Convert to Binary (Thresholding)
- 3.4 Box Filter (Custom Kernel)
- 3.5 Smoothing Filters
- 3.6 Complete Python Example

1. Image Basics

Before writing any vision algorithm, we need to understand what a digital image actually is. At its core, a digital image is simply a 2D grid (matrix) of numbers. Each cell in the grid is called a pixel (picture element). The number stored in each pixel represents the brightness or colour at that location.

1.1 Image Types Overview

Type	Pixel Values	Used For
Binary	0 (black) or 1 (white) only	Text, silhouettes, masks, thresholded images
Grayscale	Integer 0–255 (8-bit); 0 = black, 255 = white	Medical imaging, face detection, filtering
Color (RGB)	3 channels, each 0–255 (R, G, B)	Most photographs, video, object recognition

1.2 Binary Images

In a binary image, every pixel is either 0 (black) or 1 (white). There are no intermediate shades. The image is a 2D grid of width p and height q :

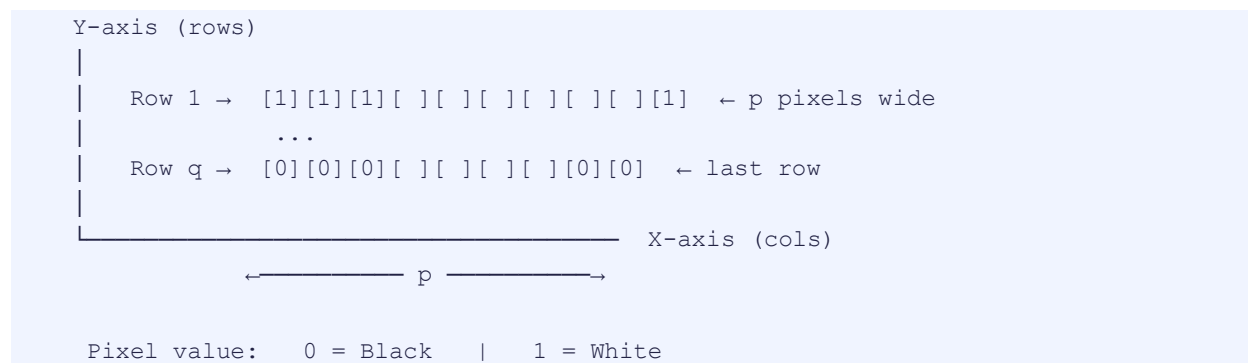


Figure 7 — Binary Image Structure ($p \times q$ grid, each cell = 0 or 1)

Binary images are produced by thresholding a grayscale image: pixels brighter than a chosen threshold become 1 (white); darker pixels become 0 (black). Common uses include:

- Document scanning — separating printed text from white paper
- Object silhouettes — isolating a foreground object from background
- Edge masks — showing where edges are detected

1.3 Grayscale Images

A grayscale image stores one number per pixel representing brightness, from 0 (pure black) to 255 (pure white). The image dimensions are H rows $\times$ W columns, forming a 2D matrix.

```

Grayscale image — each cell holds one number

[10][ 5][ 9][ ...]
[42][87][63][ ...]      ← pixel (row, col)
[200][190][185][...]
[ ...]

Scale:  0 ————— 255
        (black)         (white)

```

Figure 8 — Grayscale Image as a 2D Matrix of intensity values 0–255

Key points about grayscale images:

- Memory: 1 byte per pixel (8 bits). A 1920 $\times$ 1080 grayscale image = ~2 MB.
- In OpenCV (Python), grayscale images are stored as NumPy arrays with shape (H, W).
- You can add a single colour channel (e.g. Red) to a grayscale image to produce a coloured version — the image looks red-tinted, with brightness from the grayscale values.

1.4 Color Images (RGB)

A colour image stores three numbers per pixel — one for each colour channel: Red, Green, and Blue. The full image is a 3D matrix of shape (H, W, 3).

```
Full color image = Red channel + Green channel + Blue channel
```

```
Each channel is a separate 2D grayscale image (H $\times$ W):
```

```
Red channel:    how much red light at each pixel  (0–255)
```

```
Green channel:  how much green light at each pixel (0–255)
```

```
Blue channel:   how much blue light at each pixel  (0–255)
```

```
Example pixel: (R=255, G=100, B=0) → orange colour
```

```
Example pixel: (R=0,   G=0,   B=255) → pure blue
```

```
Example pixel: (R=0,   G=0,   B=0)   → black
```

```
Example pixel: (R=255, G=255, B=255) → white
```

Figure 9 — Color Image = three stacked grayscale channels (R, G, B)

□ OpenCV vs Standard RGB Order

OpenCV stores colour images in BGR order (Blue-Green-Red), NOT the standard RGB order. This is a common source of bugs! When displaying with matplotlib (which expects RGB), swap the channels: `img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)` `plt.imshow(img_rgb)`

1.5 Image Histograms

An image histogram shows how pixel intensities are distributed across the image. It answers the question: 'How many pixels have brightness = 0? How many have brightness = 100? How many have brightness = 255?'

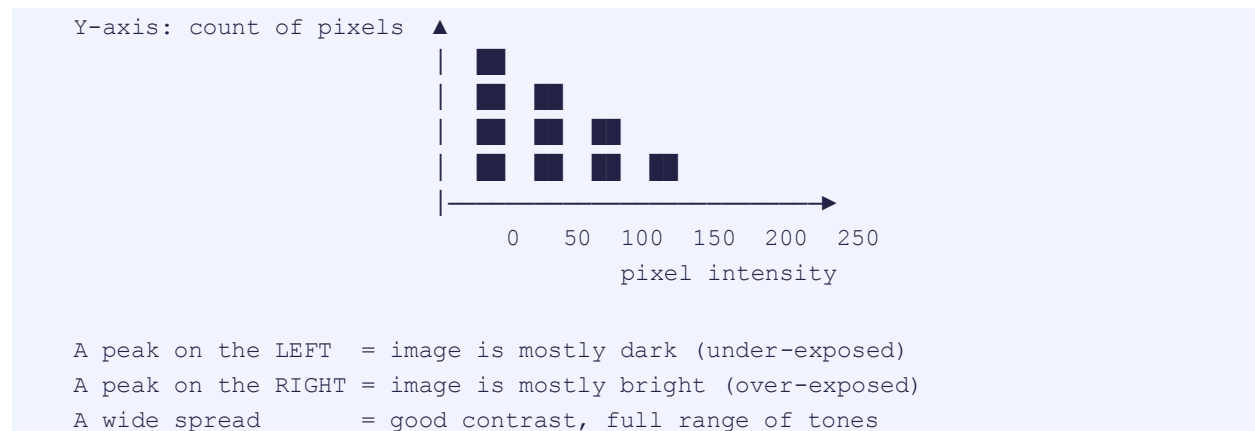


Figure 10 — Image Histogram showing pixel intensity distribution

Histograms are built from bins — each bin groups a range of intensities. A histogram with 256 bins (one per intensity value) gives the most detail.

Why histograms matter:

- Exposure analysis — Is the image too dark or too bright?
- Histogram equalisation — Spread intensities to improve contrast automatically
- Image comparison — Two images of the same scene should have similar histograms
- Segmentation — Separate foreground from background using histogram valleys as thresholds

2. Image Noise

2.1 What is Noise?

Whenever we capture an image, the recorded pixel values are never perfectly accurate — random errors are introduced by the physical process of capturing light and converting it to digital numbers. These random errors are called noise.

□ Real-World Analogy

Imagine you are measuring a tabletop with a ruler, but your hand shakes slightly each time. You measure the same edge 10 times and get slightly different numbers each time (e.g., 30.1, 29.8, 30.3, 30.0...). The true length is 30 cm, but each measurement has a tiny random error added. Image noise works exactly the same way — every pixel value has a small random error added on top of the true brightness.

2.2 Sources of Noise

Noise in digital images comes from several physical sources:

- Light Variations — fluctuations in the light source (flickering, shadows)
- Camera Electronics — thermal noise in the sensor's electronic circuits (shot noise, read noise)
- Surface Reflectance — uneven surface textures reflecting light irregularly
- Lens Imperfections — optical aberrations, dust on the lens

2.3 Mathematical Model of Noise

We model noise mathematically as an additive random term on top of the true pixel value:

$$\hat{I}(x, y) = I(x, y) + n(x, y)$$

Where:

- $\hat{I}(x, y)$ — the noisy image we actually observe (what the camera records)
- $I(x, y)$ — the true ideal pixel value (what we wish we could record)
- $n(x, y)$ — the noise term: a random number, different for each pixel

True Image I		Noise n		Observed Image $\hat{I}$
$\begin{bmatrix} 90 & 90 & 90 \\ 90 & 0 & 90 \\ 90 & 90 & 90 \end{bmatrix}$	+	$\begin{bmatrix} +3 & -2 & +7 \\ -5 & +1 & +4 \\ +2 & -4 & -1 \end{bmatrix}$	=	$\begin{bmatrix} 93 & 88 & 97 \\ 85 & 1 & 94 \\ 92 & 86 & 89 \end{bmatrix}$

(clean values) (random errors) (what camera sees)

Figure 11 — Noise is added to every pixel independently

2.4 Gaussian Noise — The Most Common Type

The most common noise model is Gaussian noise, where the noise values $n(x, y)$ follow a Gaussian (normal) distribution — the familiar bell-curve shape. This means:

- Most noise values are small (close to zero)
- Large noise values (very bright or very dark errors) are rare
- Positive and negative errors are equally likely (zero mean)

The mathematical formula for the Gaussian noise distribution:

$$n(x, y) = e^{(-n^2 / 2\sigma^2)}$$

where σ (sigma) controls the width of the bell curve (= noise strength)

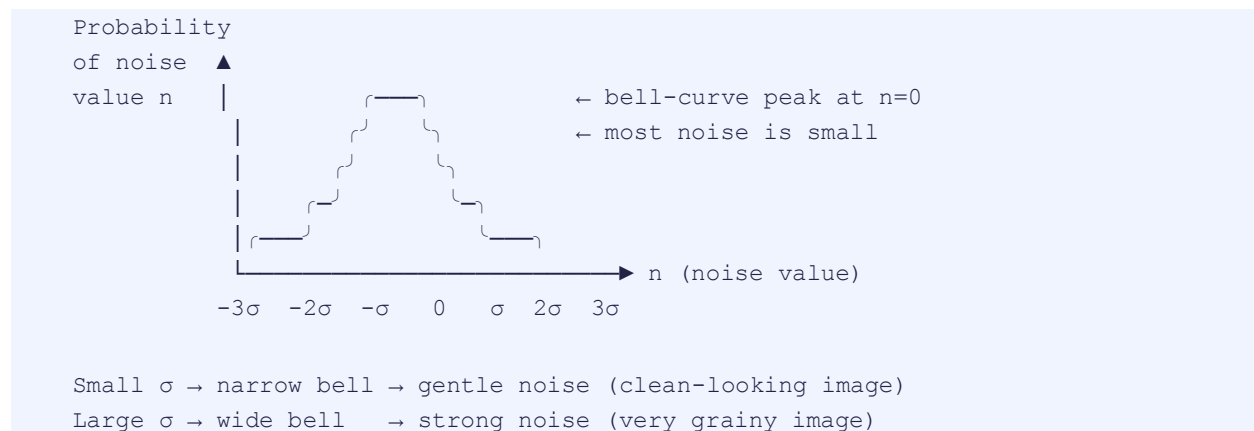


Figure 12 — Gaussian Noise Bell Curve. σ controls how noisy the image looks

σ (sigma) value	Effect on Image
$\sigma = 5$	Very mild noise — image looks almost clean
$\sigma = 25$	Moderate noise — visible grain (like a photo taken in low light)
$\sigma = 50$	Heavy noise — image details hard to see
$\sigma = 100+$	Extreme noise — image barely recognisable

3. OpenCV Functions in Python

Now let us look at the actual Python code using OpenCV to perform all the operations discussed above. Each code block is explained step-by-step.

3.1 Sample Program: Read and Display an Image

The simplest OpenCV program loads an image from disk and displays it in a window:

```
import cv2                                # import OpenCV
import numpy as np                        # import NumPy (for arrays)

# — Load an image from disk —————
# cv2.imread() reads the image file and returns a NumPy array
# The second argument:
#   cv2.IMREAD_COLOR   (=1) → load as colour (BGR, 3 channels)
#   cv2.IMREAD_GRAYSCALE (=0) → load as grayscale (1 channel)
#   cv2.IMREAD_UNCHANGED (-1) → load as-is including alpha
image = cv2.imread('lena.jpg', cv2.IMREAD_COLOR)

# — Check if loading succeeded —————
# If the file was not found, imread returns None
if image is None:
    print('Error: Image not found!')
    exit(-1)

# — Print image information —————
print('Shape:', image.shape)      # e.g. (512, 512, 3) → H x W x channels
print('dtype:', image.dtype)     # e.g. uint8 → values 0-255

# — Display the image in a window —————
cv2.namedWindow('My Window', cv2.WINDOW_NORMAL) # create resizable window
cv2.imshow('My Window', image)                 # show image
cv2.waitKey(0)                                 # wait for any key press
cv2.destroyAllWindows()                         # close all windows
```

3.2 Convert a Colour Image to Grayscale

Use `cv2.cvtColor()` to convert between colour spaces. BGR→GRAY computes a weighted average of the three channels (the same weighting the human eye uses):

```
import cv2
```

```
# Load colour image
img_color = cv2.imread('photo.jpg', cv2.IMREAD_COLOR)

# — Convert to Grayscale —————
# Formula used internally: Gray = 0.114*B + 0.587*G + 0.299*R
# (green gets most weight because human eyes are most sensitive to green)
img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

print('Color shape:', img_color.shape) # (H, W, 3)
print('Gray shape:', img_gray.shape)   # (H, W) ← only 2D!

cv2.imshow('Color', img_color)
cv2.imshow('Grayscale', img_gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3.3 Convert to Binary Image (Thresholding)

Thresholding converts a grayscale image into binary: pixels above the threshold → white (255), pixels below → black (0).

```
import cv2

img_gray = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)

# — Simple Threshold —————
# Arguments: (source, threshold_value, max_value, type)
# threshold_value = 120 → pixels > 120 become max_value (255)
# pixels <= 120 become 0 (black)
ret, img_binary = cv2.threshold(img_gray, 120, 255, cv2.THRESH_BINARY)

print('Threshold used:', ret) # returns the actual threshold value

# — Otsu's Method (automatic threshold selection) —————
# Otsu's algorithm automatically finds the BEST threshold value
# by minimising within-class variance (no manual guessing needed!)
ret_otsu, img_otsu = cv2.threshold(
    img_gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU # combine flags with +
)
print('Otsu threshold found:', ret_otsu)
```

```
cv2.imshow('Grayscale', img_gray)
cv2.imshow('Binary T=120', img_binary)
cv2.imshow('Otsu Binary', img_otsu)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3.4 Creating and Applying a Custom Box Filter (Kernel)

A box filter (also called averaging filter) replaces each pixel with the average of its neighbours. We create the kernel as a NumPy array and apply it with `cv2.filter2D()`.

```
import cv2
import numpy as np

img = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)

# — Create a 3x3 box filter kernel —————
# All 9 values = 1/9 so the sum of all kernel values = 1
# This ensures average brightness is preserved
kernel_3x3 = np.ones((3, 3), dtype=np.float32) / 9

print('3x3 Box Kernel:')
print(kernel_3x3)
# Output:
# [[0.111 0.111 0.111]
#  [0.111 0.111 0.111]
#  [0.111 0.111 0.111]]

# — Apply the kernel using filter2D —————
# Arguments: (src, ddepth, kernel, anchor, delta, borderType)
# ddepth = -1 means output has same depth as input
# anchor = (-1,-1) means kernel centre is used as anchor
# delta = 0 means no offset added to output
# borderType = cv2.BORDER_DEFAULT (reflect border)
result = cv2.filter2D(
    src=img,
    ddepth=-1,
    kernel=kernel_3x3,
    anchor=(-1, -1),
    delta=0,
    borderType=cv2.BORDER_DEFAULT
```

```

)

# Try a larger 5x5 box filter (more blurring)
kernel 5x5 = np.ones((5, 5), dtype=np.float32) / 25
result 5x5 = cv2.filter2D(img, -1, kernel 5x5)

cv2.imshow('Original', img)
cv2.imshow('Box 3x3', result)
cv2.imshow('Box 5x5', result_5x5)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

3.5 Smoothing Filters: Box, Gaussian, Median

OpenCV provides dedicated functions for the three most common smoothing operations. All three reduce noise but in different ways:

```

import cv2
import numpy as np

img = cv2.imread('photo.jpg', cv2.IMREAD_GRAYSCALE)

# — 1. Box Filter (cv2.blur) —————
# Simplest: every pixel in the neighbourhood contributes equally
# ksize = (width, height) of the filter window – must be positive odd
blur_3 = cv2.blur(img, (3, 3)) # 3x3 neighbourhood
blur_5 = cv2.blur(img, (5, 5)) # 5x5 neighbourhood (more blurring)
blur_9 = cv2.blur(img, (9, 9)) # 9x9 neighbourhood (heavy blurring)

# — 2. Gaussian Filter (cv2.GaussianBlur) —————
# Weights pixels by distance from centre (bell-curve weights)
# Closer pixels get more weight → better edge preservation
# sigmaX = standard deviation. sigmaX=0 → auto-calculated from ksize
gauss_3 = cv2.GaussianBlur(img, (3, 3), sigmaX=0) # gentle
gauss_5 = cv2.GaussianBlur(img, (5, 5), sigmaX=1) # moderate
gauss_9 = cv2.GaussianBlur(img, (9, 9), sigmaX=2) # strong

# — 3. Median Filter (cv2.medianBlur) —————
# Replaces each pixel with the MEDIAN (middle value) of its neighbourhood
# Excellent for Salt-and-Pepper noise (random black/white pixels)
# ksize must be odd: 3, 5, 7 ...
median_3 = cv2.medianBlur(img, 3) # 3x3 neighbourhood

```

```

median_5 = cv2.medianBlur(img, 5)    # 5x5 neighbourhood

# — Show all results —————
cv2.imshow('Original',      img)
cv2.imshow('Box 3x3',      blur_3)
cv2.imshow('Box 5x5',      blur_5)
cv2.imshow('Gaussian 3x3', gauss_3)
cv2.imshow('Gaussian 5x5', gauss_5)
cv2.imshow('Median 5x5',   median_5)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Filter	Best For	Limitation
Box (cv2.blur)	Fast, general-purpose smoothing	Can create ringing artefacts
Gaussian (cv2.GaussianBlur)	Natural-looking smooth blur; best general filter	Slightly slower than box filter
Median (cv2.medianBlur)	Salt-and-pepper noise (random black/white pixels)	Slower for large kernels; non-linear

3.6 Complete Python Example: Add Noise and Apply All Filters

This complete script adds Gaussian noise to an image and compares all three smoothing filters side-by-side using matplotlib:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# — 1. Load image —————
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
if img is None:
    # If no image available, generate a simple test pattern
    img = np.zeros((200, 200), dtype=np.uint8)
    img[50:150, 50:150] = 200    # white rectangle

# — 2. Add Gaussian noise manually —————
sigma = 25                                # noise strength
noise = np.random.normal(0, sigma, img.shape) # bell-curve random values
noisy = np.clip(                           # keep values in 0-255 range
    img.astype(np.int16) + noise.astype(np.int16),
    0, 255)

```

```
.astype(np.uint8)

# — 3. Apply all three filters —————
filtered_box = cv2.blur(noisy, (5, 5))
filtered_gauss = cv2.GaussianBlur(noisy, (5, 5), sigmaX=1)
filtered_median = cv2.medianBlur(noisy, 5)

# — 4. Compare side by side using matplotlib —————
fig, axes = plt.subplots(1, 5, figsize=(20, 4))

axes[0].imshow(img, cmap='gray'); axes[0].set_title('Original')
axes[1].imshow(noisy, cmap='gray'); axes[1].set_title(f'Noisy
( $\sigma=\{\text{sigma}\}$ )')
axes[2].imshow(filtered_box, cmap='gray'); axes[2].set_title('Box 5×5')
axes[3].imshow(filtered_gauss, cmap='gray'); axes[3].set_title('Gaussian 5×5')
axes[4].imshow(filtered_median, cmap='gray'); axes[4].set_title('Median 5×5')

for ax in axes: ax.axis('off') # hide x/y axis ticks

plt.suptitle('Image Noise and Smoothing Filters', fontsize=14)
plt.tight_layout()
plt.savefig('noise_comparison.png', dpi=150)
plt.show()

# — 5. Compute and show histogram —————
hist_original = cv2.calcHist([img], [0], None, [256], [0, 256])
hist_noisy = cv2.calcHist([noisy], [0], None, [256], [0, 256])

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(hist_original, color='blue')
plt.title('Histogram: Original Image')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Number of pixels')

plt.subplot(1, 2, 2)
plt.plot(hist_noisy, color='red')
plt.title('Histogram: Noisy Image')
plt.xlabel('Pixel Intensity (0-255)')
plt.ylabel('Number of pixels')

plt.tight_layout()
plt.savefig('histograms.png', dpi=150)
```

```
plt.show()
```

Summary

Lectures 1 & 2 — Key Concepts at a Glance

Computer Vision:

- Takes images as input → uses geometry, physics, learning → outputs useful information
- Applications: OCR, surveillance, medical imaging, 3D reconstruction, self-driving cars

Image Types:

- Binary: 0 or 1 per pixel | Grayscale: 0–255 | Color (RGB): three channels 0–255 each

Image Noise:

- Formula: $\hat{I}(x,y) = I(x,y) + n(x,y)$ — true value + random noise
- Gaussian noise: bell-curve distribution, σ controls strength

OpenCV Functions:

- `cv2.imread()` / `cv2.imshow()` — load and display images
- `cv2.cvtColor()` — convert between colour spaces (BGR, Gray, HSV...)
- `cv2.threshold()` — convert grayscale to binary
- `cv2.blur()` — box/averaging filter
- `cv2.GaussianBlur()` — Gaussian filter (best general-purpose smoother)
- `cv2.medianBlur()` — Median filter (best for salt-and-pepper noise)

Next Lecture: Image Filtering in depth — correlation, convolution, Gaussian kernels, and edge detection foundations.